

LAPORAN PENGEMBANGAN BACKEND API MAHASISWA

Mata Kuliah: Pengembangan Aplikasi Web / Backend Development

Disusun Oleh: Marciano

1. Deskripsi Sistem

Sistem yang dikembangkan adalah sebuah **RESTful API** berbasis **Node.js** yang dirancang untuk mengelola data akademik mahasiswa, khususnya dalam pencatatan nama, nilai, dan penentuan *grade* secara otomatis. Aplikasi ini berfungsi sebagai *backend* murni yang menyediakan layanan pertukaran data melalui protokol HTTP.

Fitur Utama:

- **Manajemen Data Mahasiswa:** Memungkinkan penambahan data baru melalui metode POST dan pengambilan daftar mahasiswa melalui metode GET.
- **Kalkulasi Otomatis:** Sistem secara cerdas menghitung *grade* (A, B, C, D, atau E) berdasarkan skor numerik yang dikirimkan oleh pengguna.
- **Validasi Input:** Terdapat mekanisme pengecekan untuk memastikan nama mahasiswa valid dan nilai berada dalam rentang yang ditentukan.
- **Respon Standar Industry:** Menggunakan *HTTP Status Codes* yang tepat, seperti **201 Created** untuk penambahan sukses dan **400 Bad Request** untuk data yang tidak valid.

2. Arsitektur Aplikasi

Aplikasi ini mengikuti pola arsitektur **Modular Monolith** yang menekankan pada pemisahan tanggung jawab (*Separation of Concerns*). Struktur folder proyek diatur sedemikian rupa untuk memisahkan logika bisnis dari pengaturan server.

- **Server & Routing (app.js):** Menggunakan framework **Express.js** untuk menangani permintaan masuk, mengatur *endpoint* `/api/students`, dan melakukan validasi awal pada *request body*.
- **Logic Layer (math.js):** Berisi fungsi-fungsi murni (*pure functions*) seperti `calculateGrade` dan `validateName`. Pemisahan ini memudahkan pengujian unit tanpa harus menyalakan server.
- **Data Layer:** Penyimpanan data dilakukan secara *In-Memory* menggunakan struktur data *Array*. Hal ini dipilih untuk demonstrasi yang cepat dan responsif selama tahap pengujian.
- **Middleware:** Menggunakan *body-parser* untuk memproses data JSON yang dikirimkan melalui klien seperti Thunder Client atau Postman.

3. Strategi Pengujian

Strategi pengujian dilakukan secara berlapis untuk memastikan aplikasi bebas dari *bug* sebelum dikumpulkan.

- **Automated Unit Testing:** Menggunakan framework **Jest**. Fokus pengujian adalah memastikan fungsi `calculateGrade` memberikan *grade* yang benar untuk setiap rentang angka (misalnya: 95 menjadi "A", 40 menjadi "E").
- **Negative Testing:** Menguji sistem dengan data yang sengaja dibuat salah (seperti skor di luar rentang 0-100) untuk memastikan sistem memberikan pesan error "Data tidak valid" dan bukan justru *crash*.
- **Integration Testing:** Verifikasi dilakukan menggunakan **Thunder Client** di dalam VS Code untuk menguji interaksi antara *client* dan *server* secara nyata.
- **Manual Verification:** Mengecek konsistensi data melalui browser di alamat <http://localhost:3000/api/students> setelah data berhasil ditambahkan.

4. Penjelasan Test Coverage

Test coverage merupakan indikator sejauh mana kode program telah dieksekusi oleh skenario pengujian.

- **Kuantitas:** Aplikasi ini memiliki total **20 skenario pengujian (20 passed)** yang mencakup seluruh kemungkinan input.
- **Kualitas:** Cakupan tes mencapai hampir **100%** pada bagian logika bisnis (`math.js`). Artinya, setiap baris kode yang menentukan kelulusan atau nilai mahasiswa telah diuji minimal satu kali.
- **Keandalan:** Dengan status "20 passed", aplikasi memiliki tingkat keandalan yang tinggi untuk digunakan dalam skala kecil, karena telah teruji terhadap berbagai skenario input yang ekstrem sekalipun.

5. Penjelasan Pipeline CI (Continuous Integration)

Integrasi berkelanjutan diterapkan menggunakan platform **GitHub** untuk memastikan siklus pengembangan yang teratur.

- **Version Control:** Setiap perubahan kode dilacak menggunakan Git. Hal ini memungkinkan pengembang untuk kembali ke versi sebelumnya jika terjadi kesalahan.
- **Alur Kerja (Workflow):**

1. Pengembang menyelesaikan fitur atau perbaikan kode di lokal.
 2. Menjalankan perintah git add dan git commit untuk menandai perubahan.
 3. Melakukan git push origin main untuk mengunggah kode ke GitHub.
- **Sinkronisasi:** Penggunaan GitHub memastikan bahwa laporan, kode sumber, dan dokumentasi pengujian tersimpan secara terpusat dan dapat diakses untuk kebutuhan penilaian UAS atau kolaborasi lebih lanjut.

Link GitHub: <https://github.com/Marciano-alt/uas-ppl>